

TRABAJO PRÁCTICO NRO 02 - GESTIÓN COLABORATIVA, CONTROL DE VERSIONES Y ORGANIZACIÓN EMPRESARIAL

Análisis de datos climáticos con Python, GitHub, Jira y Google Colab

Alumno: Leandro Nicolás Acuña

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional

Materia: Organización Empresarial - M26 C1-17 - 1er Cuatr.

Coordinadora / docente titular: Gabriela Martínez

Profesor asignado: Guillermo Caryl

Comisión: 16-17 | Aula: 1C-17 | Modalidad: Asíncrono

TABLA DE CONTENIDO

1. Introducción
2. Objetivos del trabajo
3. Organización del proyecto y roles
4. Gestión en Jira y trazabilidad
5. Implementación técnica en GitHub y Colab
6. Resultados del análisis climático
7. Seguridad y buenas prácticas
8. Conclusión y referencias

1. INTRODUCCIÓN

En este trabajo práctico se desarrolló un proyecto simple de análisis de datos climáticos, usando Python como lenguaje principal y aplicando herramientas de gestión y control de versiones. La idea principal fue no hacer solamente un script, sino también dejar registro de como se organiza el trabajo, como se suben los cambios y como se puede revisar lo que hizo cada parte.

El escenario elegido fue el de clima. Para eso se usó un dataset público de Open-Meteo con datos diarios de Buenos Aires durante el año 2024. Con esos datos se calcularon indicadores básicos como temperatura promedio, temperatura máxima, temperatura mínima y precipitaciones. También se generó un gráfico para ver la evolución mensual.

Como el TP lo hice de manera individual, tuve que adaptar la consigna de los roles. En vez de repartir el trabajo con otros integrantes, tomé los tres roles pedidos por la actividad: organizador, desarrollador y revisor. Me pareció una forma ordenada de cumplir el flujo sin inventar compañeros que no participaron.

2. OBJETIVOS DEL TRABAJO

- Aplicar Git y GitHub para controlar versiones del proyecto.
- Usar Jira para planificar tareas y relacionarlas con commits.
- Trabajar con una estructura clara de carpetas: datos, scripts y resultados.
- Crear un script reproducible en Python, pensado para ejecutarse también en Google Colab.
- Cuidar la seguridad del repositorio, evitando subir tokens o archivos innecesarios.

3. ORGANIZACIÓN DEL PROYECTO Y ROLES

La consigna propone una célula de desarrollo con Hugo, Paco y Luis. En mi caso lo resolví solo, entonces usó esos roles como una manera de organizarme. No es que haya tres personas reales, sino tres responsabilidades distintas dentro del mismo trabajo.

Rol	Responsabilidad tomada	Acción realizada
P1 - Organizador	Preparar la base del proyecto	Crear repo, carpetas, README y archivo .gitignore.
P2 - Desarrollo	Resolver la parte técnica	Crear el script Python para procesar datos climáticos.
P3 - Revisión	Controlar calidad y seguridad	Revisar documentación, tokens, rutas relativas y resultados.

Repositorio de GitHub: <https://github.com/Leanmucho/TP-2-Gesti-n-Colaborativa-Control-de-Versiones-y-Organizaci-n-Empresarial>

Tablero de Jira: <https://acunaleandro3.atlassian.net>

4. GESTIÓN EN JIRA Y TRAZABILIDAD

Para cumplir con la parte organizacional, la planificación se deja separada en tres issues. La clave sugerida para el proyecto es TP2, porque es corta y fácil de usar en los commits. Cada tarea representa una parte del trabajo y permite mostrar trazabilidad entre Jira y GitHub.

Issue	Título	Estado esperado
TP2-1	Crear repositorio y estructura inicial	Finalizado
TP2-2	Desarrollar análisis climático en Python	Finalizado
TP2-3	Revisar documentación y seguridad	Finalizado

Los commits se deben escribir con el ID del issue al comienzo, por ejemplo: TP2-1: crear estructura inicial del repositorio. Esta regla ayuda a que no quede separado lo que se planificó en Jira de lo que después se subió a GitHub.

- TP2-1: crear estructura inicial del repositorio
- TP2-2: agregar análisis climático en Python
- TP2-3: documentar revisión y buenas prácticas

5. IMPLEMENTACIÓN TÉCNICA EN GITHUB Y COLAB

El repositorio se organizó con las carpetas pedidas por la consigna. La carpeta datos contiene el CSV, scripts contiene el programa en Python y resultados guarda los archivos generados al ejecutar el análisis. Esto hace que el proyecto sea más fácil de revisar y reproducir.

Carpeta o archivo	Uso dentro del proyecto
datos/	Guarda el dataset clima_buenos_aires_2024.csv.
scripts/	Contiene analisis_clima.py, que procesa el dataset.
resultados/	Guarda resúmenes CSV y el gráfico generado.
README.md	Explica el proyecto, fuente de datos y ejecución.
.gitignore	Evita subir archivos temporales o sensibles.

En Google Colab se debe configurar primero la identidad de Git, con el nombre y correo del alumno. Después se clona el repositorio y se ejecuta el script con `python scripts/analisis_clima.py`. Para subir cambios desde Colab se usa un Personal Access Token, pero no se debe dejar escrito en celdas públicas ni subirlo al repositorio.

El flujo recomendado es trabajar en una rama `feature/analisis-clima`, subirla a GitHub y abrir un Pull Request hacia main. Aunque sea una entrega individual, se pueden dejar comentarios técnicos en el PR para mostrar la Revisión, por ejemplo sobre rutas relativas y seguridad del token.

6. RESULTADOS DEL ANÁLISIS CLIMÁTICO

El dataset utilizado contiene registros diarios de Buenos Aires durante 2024. El script toma esos datos, calcula indicadores generales y después agrupa la información por mes. Los resultados principales fueron los siguientes:

Indicador	Resultado
Temperatura promedio	17.68 °C
Temperatura máxima	36.10 °C
Temperatura mínima	-1.40 °C
Promedio diario de precipitación	3.00 mm
Precipitación total anual	1096.80 mm

Estos valores permiten tener una mirada general del clima durante el año. El dato que más me llamó la atención fue la diferencia entre la temperatura mínima y máxima, porque muestra que el dataset tiene bastante variación entre estaciones.

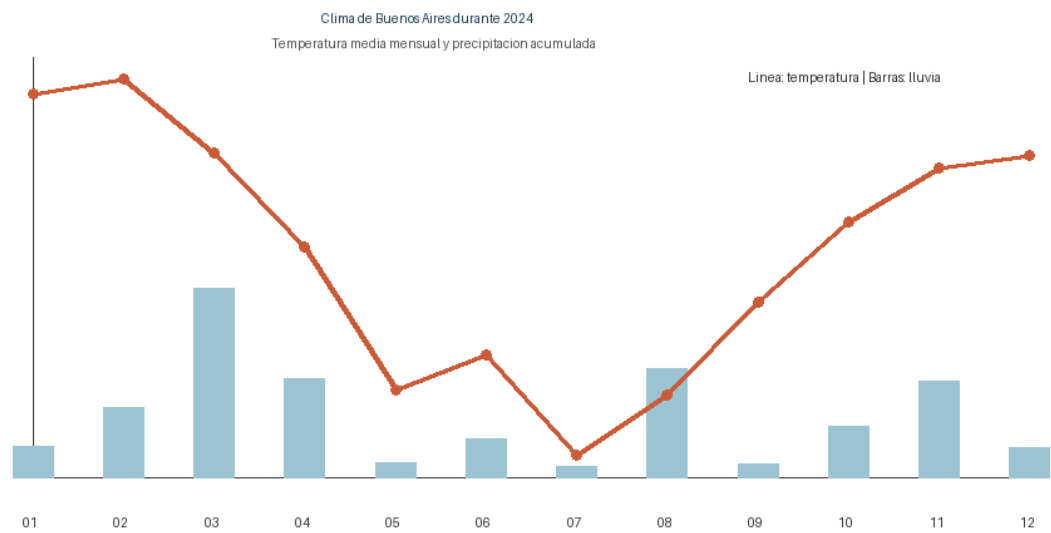


Figura 1. Evolución mensual de temperatura media y precipitaciones en Buenos Aires durante 2024.

El gráfico ayuda a ver la forma general del año: meses más cálidos al comienzo y al final, y temperaturas más bajas en la mitad del año. Las barras muestran la precipitación mensual acumulada.

7. SEGURIDAD Y BUENAS PRÁCTICAS

Una parte importante del trabajo fue cuidar que el repositorio no tenga información sensible. Por eso se agregó un archivo `.gitignore` y se aclara que el token de GitHub no se pega en el código. Si se llegara a exponer un token por error, lo correcto sería revocarlo desde GitHub y crear uno nuevo.

- No subir tokens, claves ni archivos `.env`.
- Usar rutas relativas para que el código funcione en Colab y no dependa de mi PC.
- Guardar datos, código y resultados en carpetas separadas.
- Hacer commits frecuentes y con ID de Jira.

- Revisar el Pull Request antes del merge final.

8. CONCLUSIÓN

Con este TP pude practicar no solo Python, sino también la parte organizativa del desarrollo. Antes veía GitHub más como un lugar donde se suben archivos, pero con este trabajo queda más claro que También sirve para registrar decisiones, revisar cambios y ordenar un proyecto.

También me sirvió para entender que Jira y Git no son cosas separadas: si los commits están conectados con los issues, después es mucho más fácil saber por qué se hizo cada cambio. En una empresa o equipo real esto sería importante para no perder el seguimiento del trabajo.

Use asistencia de IA como apoyo para ordenar ideas y revisar la redacción, pero el contenido fue adaptado a mi caso: entrega individual, escenario de clima, repo propio y forma simple de explicar el proceso. No copié una respuesta genérica porque la idea era que el informe quede entendible y con mis datos reales.

REFERENCIAS

Open-Meteo. (2024). Historical Weather API. <https://open-meteo.com/>

Atlassian. (s.f.). Jira Software. <https://www.atlassian.com/software/jira>

GitHub Docs. (s.f.). GitHub documentation. <https://docs.github.com/>

Tecnicatura Universitaria en Programación. Organización Empresarial. Trabajo práctico: Gestión Colaborativa, Control de Versiones y Organización Empresarial. UTN, 2026.